

Sintaxis - ¿Cómo se especifica un lenguaje de programación?

Material de lectura sobre el tema: Complementar lo presentado en este apunte con:

- Sebesta: Capítulo 3 (hasta la sección 3.4).
- Scott: Capítulo 2 (Introducción y sección 2.1), y Capítulo 4 (Introducción y secciones 4.1 y 4.2).

Tengan en cuenta que el libro de Scott hace una presentación del tema que está más asociada al desarrollo del traductor del lenguaje que a la parte conceptual, por lo tanto manejen el nivel de profundidad en la lectura. Ante cualquier duda, consultar.

Algunos detalles que cuesta encontrar en los libros o simplemente no están

La especificación precisa de un lenguaje de programación es esencial para describir el comportamiento computacional del mismo. Sin una notación clara del efecto de los constructores del lenguaje es imposible determinar dicho comportamiento. En general la especificación formal de los lenguajes es útil a varios propósitos:

- ✧ Ayuda a la comprensión del lenguaje.
- ✧ Brinda soporte para la estandarización de lenguajes.
- ✧ Guía en el diseño de lenguajes.
- ✧ Ayuda al sistema de escritura del compilador y lenguaje.
- ✧ Soporta la verificación de correctitud de los programas.
- ✧ Modela la especificación de software.

Un lenguaje de programación es un lenguaje formal (como los que vieron en las primeras materias de teoría en la carrera), solo que es mucho más complejo que los lenguajes que utilizaron en las prácticas de esas materias.

Recordemos que un **lenguaje formal** es un conjunto de cadenas de caracteres elegidas de un conjunto finito y fijo de símbolos (*alfabeto*). Las cadenas que pertenecen al lenguaje, son llamadas *frases* o *constructores*. Si uno de concentra en esta definición y la asocia a lo estudiado en Teoría de la Computabilidad y Lenguajes Formales y Autómatas con respecto a los tipos de los lenguajes, ¿cuál sería el tipo de cualquier lenguaje de programación?

El tipo formal sería Tipo 0 o estructurado por frases en la jerarquía de Chomsky. Si el lenguaje fuera de un tipo menor en la jerarquía de Chomsky habría problemas computacionales que no se podrían expresar en ellos.

Ahora bien, este detalle genera un problema computacional serio ya que los lenguajes de tipo cero son recursivos enumerables, es decir no tienen un reconocedor que siempre termine. Dejando la complejidad y eficiencia de lado, podríamos hacer un “compilador” que en caso de que nuestro programa pertenezca al lenguaje nos responda afirmativamente, pero en caso de no pertenecer puede ser que nos responda negativamente o no termine. ¿Imaginan un lenguaje con un traductor que funcione de esta manera?

La computación ha encontrado a lo largo de su historia herramientas vinculadas a lenguajes formales que apuntan a mejorar la eficiencia de los traductores. Contrariamente a la percepción que se tiene cuando se estudian los conceptos de teoría, los desarrollos teóricos no nacieron de esta manera, sino que fueron motivados por algún problema práctico que requería de una solución lo más formal posible (para garantizar su correctitud). Un ejemplo de esto son las formas normales que existen para las gramáticas libres de contexto (retomaremos este punto unos párrafos más adelante).

Volviendo al tema del tipo del lenguaje, si los lenguajes de programación son de tipo 0 y no es posible tener un reconocedor para ellos, la única estrategia viable para lidiar con esto sería descomponer el problema en subproblemas, es decir tratar de dividir la naturaleza del lenguaje en partes que sean tratables. Claramente esta solución deja fuera del reconocedor lo que hace que el lenguaje sea recursivo enumerable. Esta es la razón de que, por ejemplo, no haya reconocedores que detecten ciclos repetitivos mal definidos (problema de la detención).

La descripción de cualquier lenguaje de programación se termina dividiendo entonces en:

1. **Sintaxis:** que manipula la formación de las frases.
2. **Semántica:** que manipula el significado de las frases.
3. **Pragmatismo:** que manipula el uso práctico de las frases.

Esta división la tienen también las lenguas vivas. En el caso de los lenguajes de programación, el pragmatismo es la parte más difícil de definir ya que son cuestiones ajenas a los lenguajes en sí. Algunos libros, por ejemplo, se refieren al pragmatismo como al hecho de que el lenguaje cuente con un debugger, IDE, etc. En las lenguas vivas el pragmatismo tiene que ver con el tono con el que se dice una frase, el contexto, el gesto del interlocutor. El significado de la frase cambia de acuerdo a esto (por ejemplo, no es lo mismo decir algo en tono de pregunta, en tono irónico o riéndose). En lenguajes de programación este grado de versatilidad no existe.

Las otras dos patas de la definición, la sintaxis y la semántica, sí son fundamentales. Si un lenguaje de programación no las provee, no está correctamente definido y por lo tanto no es tratable como lenguaje de programación. No habría manera de saber cómo escribir un programa ni cuál es el significado del mismo.

La **sintaxis** se refiere a la forma, la estructura del programa. Es análogo a lo que sucede en las lenguas vivas. Se trata de un conjunto de reglas que indican la forma correcta de los programas, pero no garantizan que tengan sentido.

Volviendo a la discusión previa, podemos decir que la sintaxis debería ser de tipo 1 o sensible al contexto, ya que para este tipo de lenguajes tenemos garantizada la existencia de un reconocedor recursivo (siempre termina).

El problema que tenemos aquí es un problema de eficiencia y de costo. El traductor del lenguaje debe ser eficiente (imaginen la utilidad de un traductor si tarda mucho tiempo en indicar si el programa es sintácticamente correcto), por lo cual nuevamente se apela a una división de problemas. Surge entonces:

- ✧ **Sintaxis propiamente dicha:** es la parte de la sintaxis que puede expresarse con una gramática libre de contexto.
- ✧ **Semántica estática:** captura el resto de la sintaxis, aquellas características que pueden chequearse sin ejecutar el programa. Son aspectos que no pueden escribirse con una gramática libre de contexto (por ejemplo, controlar que una variable fue declarada antes de su uso) o cuestiones que podrían escribirse pero que complican la gramática de manera innecesaria (por ejemplo, el control de tipos en las expresiones implica la multiplicación de las reglas de expresiones por la cantidad de tipos a considerar).

La sintaxis propiamente dicha se maneja con las gramáticas libres de contexto por dos motivos:

- ✧ La legibilidad de la gramática (dada fundamentalmente por la notación BNF). Esto hace que el usuario final del lenguaje puede entender fácilmente la estructura que debe seguir el programa en ese lenguaje de programación.
- ✧ La eficiencia en la implementación del reconocedor. Se puede implementar un *parser* (ese es el nombre que tiene esta parte del proceso en los compiladores e intérpretes avanzados) para reconocer la sintaxis del lenguaje de manera extremadamente eficiente. Históricamente hay mucho trabajo hecho sobre cómo mejorar los tiempos de reconocimiento. Tengan en cuenta que cuando aparecieron los lenguajes cualquier mejora en este punto era absolutamente crítica. Al comienzo de las investigaciones aparecieron las formas normales (Chomsky y Greibach), como una forma de mejorar la eficiencia de los reconocedores de lenguajes de programación.

Recuerden que, como todo lenguaje formal, se parte de la definición de un alfabeto. La verificación de que formen palabras válidas se hace en el reconocedor con un scanner o analizador léxico. Esto es así porque de esta manera la gramática libre de contexto trabaja con tokens y el control de esta parte se puede hacer de manera más eficiente con herramientas de lenguajes regulares.

La semántica estática se formaliza a través de lo que se conoce como gramática de atributos y se implementa en el reconocedor a través del analizador semántico. La dificultad de esta etapa está altamente vinculada a las características de los lenguajes. Tiene principal relevancia en lenguajes con tipado estático.

Una gramática de atributos es un mecanismo utilizado para describir más de lo que se puede describir con la gramática libre de contexto. Es una extensión de las gramáticas libres de contexto, la idea es agregarle a la gramática tradicional, información semántica a lo largo del árbol de parsing.

De esta manera, para cada símbolo de la gramática X , hay un conjunto $A(X)$ de atributos. El conjunto se divide en dos conjuntos disjuntos $S(X)$ y $I(X)$, que definen a los atributos *sintetizados* (toman valor en las hojas y van subiendo por el árbol de derivación) y *heredados* (toman valor en algún lugar del árbol y van bajando ese valor hacia las hojas), respectivamente. En las hojas del árbol de derivación, además, hay atributos *intrínsecos*, que toman sus valores en el análisis léxico. Pueden mirar los ejemplos que aparecen en los libros. A continuación, se incluye el que está en el Sebesta.

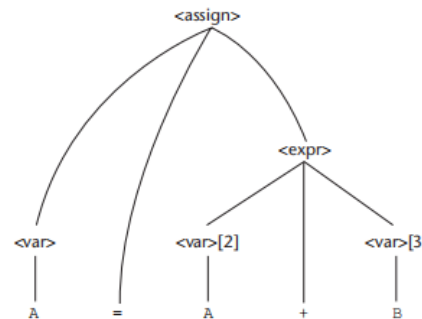
Consideren la siguiente gramática de atributos:

EXAMPLE 3.6	An Attribute Grammar for Simple Assignment Statements
1.	Syntax rule: $\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expr} \rangle$ Semantic rule: $\langle \text{expr} \rangle.\text{expected_type} \leftarrow \langle \text{var} \rangle.\text{actual_type}$
2.	Syntax rule: $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle[2] + \langle \text{var} \rangle[3]$ Semantic rule: $\langle \text{expr} \rangle.\text{actual_type} \leftarrow$ if $(\langle \text{var} \rangle[2].\text{actual_type} = \text{int})$ and $(\langle \text{var} \rangle[3].\text{actual_type} = \text{int})$ then int else real end if Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$
3.	Syntax rule: $\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle$ Semantic rule: $\langle \text{expr} \rangle.\text{actual_type} \leftarrow \langle \text{var} \rangle.\text{actual_type}$ Predicate: $\langle \text{expr} \rangle.\text{actual_type} == \langle \text{expr} \rangle.\text{expected_type}$
4.	Syntax rule: $\langle \text{var} \rangle \rightarrow A \mid B \mid C$ Semantic rule: $\langle \text{var} \rangle.\text{actual_type} \leftarrow \text{look-up}(\langle \text{var} \rangle.\text{string})$
The look-up function looks up a given variable name in the symbol table and returns the variable's type.	

Para la cadena "A = A+B" se genera el siguiente árbol de derivación o árbol sintáctico:

Figure 3.6

A parse tree for
A = A + B



Si tenemos en cuenta la gramática de atributos nos quedaría de esta manera:

Figure 3.7

The flow of attributes in the tree

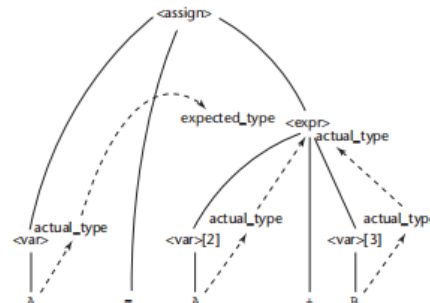
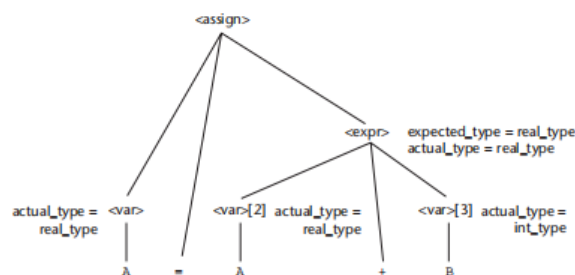


Figure 3.8

A fully attributed parse tree



Es importante tener en cuenta que si los chequeos se hacen en una sola pasada, la evaluación de los atributos y la generación del árbol sintáctico suceden al mismo tiempo. Los árboles se dibujan para poder tener una representación visual y entender que es lo que está pasando, pero, en general, en las implementaciones los árboles no se guardan o generan explícitamente a menos que sea estrictamente necesario, y cuando se generan suelen tener una minimización de nodos.

Semántica - ¿Cómo se especifica un lenguaje de programación?

Material de lectura sobre el tema: Complementar lo presentado aquí con:

- Sebesta: Capítulo 3 (sección 3.5).

La semántica de los lenguajes: un punto crítico que no es tan simple de abordar

En las páginas anteriores, discutimos las ideas escondidas detrás de la especificación de los lenguajes. Discutimos las razones que llevan a dividir el abordaje de la sintaxis con diferentes herramientas formales, a fin de garantizar la correctitud de los reconocedores de los lenguajes. Vimos que para esa parte hay un estudio interesante y cómo la teoría ha encontrado la manera de asegurar métodos que van a permitir la implementación: de scanners o analizadores léxicos, parsers o analizadores sintácticos, como así también del análisis semántico encargado de lo que definimos como semántica estática (recuerden que a pesar del nombre se trata de sintaxis, en ningún momento se apela al significado de lo que sucede en ejecución con el programa).

Sin embargo, la sintaxis no alcanza para cubrir la especificación completa del lenguaje. El significado y/o comportamiento de cada constructor del lenguaje debe incluirse en la definición de todo lenguaje de programación. La pregunta es ¿cómo hacerlo? Particularmente, ¿cómo lograrlo sin perder la garantía de correctitud del traductor? Noten que ahora usé la palabra traductor. ¿Por qué? Porque la semántica adquiere relevancia particularmente a la hora de traducir lo escrito en el lenguaje de programación al lenguaje máquina.

Lo ideal sería poder utilizar algún método de especificación formal para especificar la semántica. Como se puede ver en la bibliografía, hay varias opciones para esto, entre las que se destacan:

1. Semántica axiomática.
2. Semántica denotacional.
3. Semántica operacional.

Cada una de ellas apela a un formalismo diferente para abordar la descripción del comportamiento de los constructores del lenguaje.

La semántica **axiomática** apela a la lógica Hoare y las aserciones. Es el formalismo más “raro” para especificar semántica ya que apela más a probar que el programa es correcto que a mostrar cuál es el significado de los constructores. La semántica **denotacional** apela al formalismo de funciones recursivas parciales, mientras que la **operacional** hace lo propio pero con una máquina de estados abstracta (básicamente se basa en el formalismo de las máquinas de Turing). Claramente estas dos últimas son más “operativas” y más rigurosas en la manera de expresar la semántica, particularmente la denotacional.

La semántica denotacional ha sido utilizada para expresar la semántica de algunos lenguajes funcionales, logrando un grado de claridad y seguridad en la correctitud de las interpretaciones de los programas especificados en ellos. La vinculación entre esta semántica y los lenguajes del paradigma funcional no es casual, claramente comparten la misma base teórica formal.

Es importante tener en cuenta que los paradigmas convencionales tienen en su base de comportamiento características que atentan contra la caracterización formal del comportamiento. Por ejemplo, el uso del entorno global conduce a lo que llamamos *efectos colaterales*. Un efecto colateral es parte del cómputo y genera un cambio, pero este cambio no puede establecerse en la semántica de la operación, es un cambio que no es visible desde la parte pública de la operación y, por lo tanto, tampoco es esperado. Con esto, describirlo formalmente a través de cualquiera de las tres semánticas mencionadas es difícil.

Recuerden que:

La razón de ser de los lenguajes de programación (LP) es salvar el gap entre los programadores y el lenguaje máquina. Sin los lenguajes de programación, el programador debería hacer su trabajo de programación precisamente en lenguaje máquina. El rol de los LP es brindarles un grado de abstracción a los programadores para trabajar, pero como la máquina sigue entendiendo lo mismo que antes, es necesario que el lenguaje vaya acompañado de un traductor que se encargue de obtener la versión en código máquina de lo que el humano escribe en este nuevo dialecto.

Por esta razón, los criterios de evaluaciones de software están tan vinculados a los lenguajes de programación, sin el software del traductor no tendrían sentido. Ese software tiene dos objetivos importantes: **reconocer y traducir**.

Pueden mirar en el libro como se ven las especificaciones de la semántica de los lenguajes en cada uno de estos formalismos.

Si bien claramente estas opciones son las mejores desde el punto de vista de la correctitud de los lenguajes, son formas arduas para las cuales no siempre se amerita el esfuerzo, por detalles de los paradigmas (como por ejemplo el detalle que comentado sobre el paradigma imperativo) o por decisiones de los lenguajes en sí.

Con lo anterior, hace falta encontrar una opción a estos lenguajes formales de especificación. La primera opción que aparece entonces es la de redactar un documento (un manual de referencia del lenguaje) en el cual se explica, en lenguaje natural, el comportamiento de cada constructor. Los documentos en lenguaje natural tienen como defecto importante precisamente la falta de formalidad. El contenido de cualquier documento escrito de esta manera, por más cuidado que se tenga, no es lo suficientemente preciso y puede incluir frases que dependen de la interpretación del lector, es decir, tienen un grado de ambigüedad.

Esta solución está lejos de ser la mejor, ya que si el diseñador del lenguaje se queda solo con esto el lenguaje puede comenzar a tener dialectos. Un caso paradigmático de esto es C. El creador del lenguaje lo diseñó para su uso personal, por lo que no se detuvo a dejar la semántica lo suficientemente formalizada, y al comenzar a comercializarse las compañías encargadas de implementar los traductores interpretaron de manera diferente algunas de estas especificaciones semánticas, lo que ha llevado a tener compiladores de C con comportamientos notablemente diferentes para el mismo programa. Este detalle ha sido problemático hasta las posteriores estandarizaciones del lenguaje (C89, C90, C99, etc.).

¿Cómo se resuelve este problema entonces? Si no es posible brindar de manera completa una formalización de la semántica y hacer un documento no alcanza, ¿qué estrategia queda para no llegar a la situación de lenguajes como C? La solución es simple, pero bastante ad-hoc. **La solución de compromiso a la que llegan muchos lenguajes es la siguiente: brindan lo que pueden en lenguaje formal (a veces termina siendo nada, a veces como en lenguajes funcionales como Haskell, es todo), y el resto lo dejan en un documento de especificación, pero este documento va acompañado de una implementación.** Este último detalle es el que permite que el lenguaje quede libre de ambigüedades y puede ser estandarizado. ¿Por qué? Porque la implementación resuelve cualquier duda que pueda surgir del documento. Si hay alguna duda de cuál es el significado del constructor, la respuesta del diseñador es: “debe tener el mismo comportamiento que el traductor que acompaña la definición del lenguaje”.

¿Compilador o intérprete?

Material de lectura sobre el tema: Complementar lo presentado aquí con:

- Sebesta: Capítulo 3.
- Scott: Capítulo 4.

Es importante tener en cuenta que el alcance de este tema es solo conceptual, los detalles de cómo se implementa cada uno se profundizan en la materia "Compiladores e Intérpretes".

Elección del reconocedor/traductor del lenguaje

Hay algunas características de los lenguajes que deciden qué tipo de traductor van a necesitar, la elección generalmente escapa del control del implementador, sino que está atada a consideraciones de diseño del lenguaje. Cuanto más estático es un lenguaje, más probable es que se implemente a través de un compilador; cuanto más dinámico sea, más imposible se hace la implementación de un compilador.

Como bien saben, los compiladores están destinados a resolver todo lo que puedan antes de la ejecución, esto termina significando que están destinados a lenguajes con una semántica estática importante. Si la semántica estática no puede capturar estáticamente mucho, el compilador deja de tener sentido. ¿Qué es lo que habitualmente se captura en la semántica estática? Lo que tiene que ver con el control de tipos. De ahí el comentario anterior.

La principal diferencia en ambos como traductores está en el tradeoff entre eficiencia y flexibilidad. Los compiladores tienen la posibilidad de generar código más eficiente porque tienen disponible una versión completa del código traducido, lo que permite una manipulación del mismo para generar una versión mejorada del mismo.

Por otro lado, los intérpretes no tienen esta posibilidad, pero permiten tratar con lenguajes con una mayor flexibilidad para el programador (suelen tener características como tipado dinámico).

Los primeros intérpretes eran altamente ineficientes, postergaban todo hasta el momento de la ejecución, en la ejecución se hacía el reconocimiento, traducción y ejecución sentencia a sentencia.

Claramente no es el escenario de los intérpretes modernos. Estos intérpretes realizan el reconocimiento antes de la fase de interpretación en sí, esto es, resuelven los controles estáticos antes, permitiendo de esta manera mejorar el rendimiento en donde es posible hacerlo.

Tanto los compiladores como los intérpretes tienen fases internas bien delimitadas, las diferencias están en los momentos en los que tienen lugar. Estas son:

- i. Scanning o análisis léxico (encargado de la manipulación del archivo con el código fuente y el reconocimiento de las palabras que forman el código fuente a través de patrones léxicos, que habitualmente se expresan con expresiones regulares).
- ii. Parsing o análisis sintáctico (encargado de la sintaxis pura).
- iii. Análisis semántico (encargado de la semántica estática).
- iv. Generación de código intermedio y/o máquina.

Cada una de estas fases está ligada a los diferentes trabajos del traductor, las tres primeras corresponden al reconocimiento del código fuente y la última a la traducción.

Si bien desde el punto de vista teórico estas fases están separadas, ya que conceptualmente están abordadas a través de formalismos diferentes, a la hora de hacer una implementación, dentro de lo posible, terminan siendo módulos que trabajan colaborativamente. ¿Por qué? Nuevamente por cuestiones de eficiencia.

Resolver la implementación como una cascada de cuatro fases implica que a la salida de cada fase hay que generar una representación intermedia que sea el input de la fase siguiente, estas representaciones son grandes de tamaño y además hay que generarlas y recorrerlas en la siguiente etapa, lo que claramente es una inversión importante en uso de espacio y tiempo de ejecución. Para que se hagan una idea, si el scanning y parsing se resolvieran en modo cascada, el scanning va a recorrer todo el archivo de entrada eliminando comentarios, espacios, saltos de línea, etc. y va a pasarle al parser un nuevo archivo formado solo por tokens (el token representa qué tipo de palabra encontramos y algunos atributos que permiten tener información adicional como el lexema, línea en la que estaba ubicado en el archivo, etc.). El parser luego va a armar el árbol de derivación de esa cadena de tokens.

Esta estrategia es mucho menos eficiente que una en el cual intentemos reconocer el código fuente directamente utilizando al scanner como un módulo al cual le vamos solicitando el próximo token del archivo cada vez que haga falta.

Como les mencioné al principio del documento, los algoritmos que se utilizan para esto los van a ver con detalle en la materia “Compiladores e Intérpretes” por lo que no vale la pena profundizar al respecto aquí. Lo que sí es importante es tener una idea general de la funcionalidad de los traductores, lo que motiva esta sección del documento.